
Detección Híbrida de Secretos en Código y Pipelines

**mediante Reglas, Entropía y Aprendizaje
Automático**

Rubén Jiménez Ormad

Máster en IA Aplicada a la Ciberseguridad · Módulo 9 — TFM

Agosto de 2026 · Tutor: Fran Ramírez (Telefónica Innovation)

1. Problema y motivación

+12,8M

secretos expuestos en GitHub
en 2023 (+28 % interanual)

15.084

credenciales reales verificadas
manualmente en 818 repos

\$4,81M

coste medio de brecha
por credencial comprometida

¿Por qué ocurre?

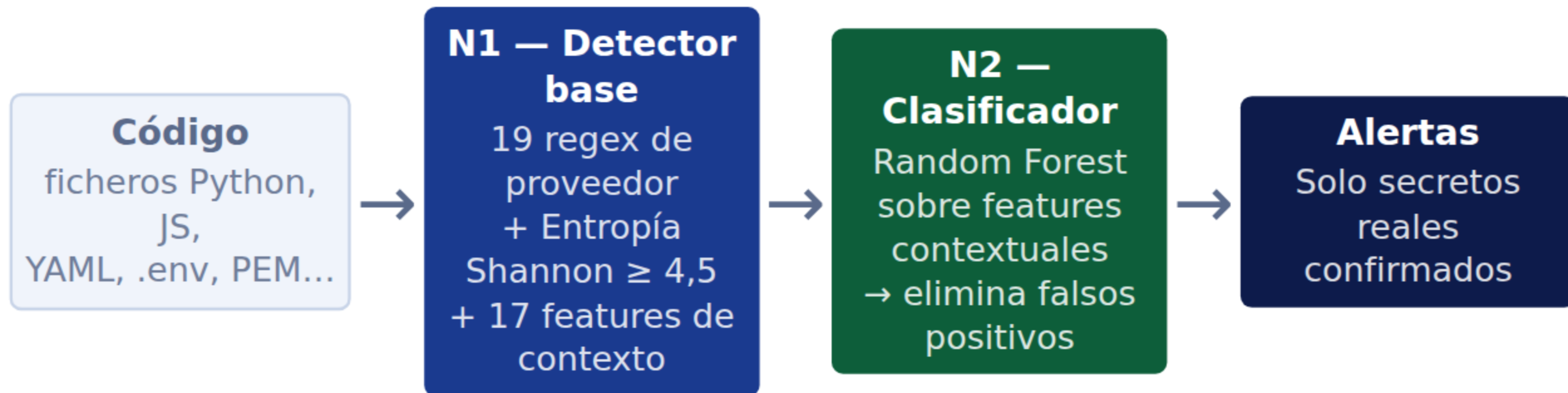
- Presión de plazos → credenciales hardcodeadas en el código
- Una vez en el historial git, **la credencial es irre recuperable sin rotación**
- Las herramientas actuales generan demasiadas falsas alertas → los equipos las ignoran

2. Estado del arte y su limitación principal

Enfoque	Herramienta típica	Ventaja	Limitación
Patrones (regex)	GitLeaks, TruffleHog	Rápido, determinista	Muchos falsos positivos en ficheros de test y docs
Entropía de Shannon	TruffleHog, detect-secrets	Detecta formatos desconocidos	UUIDs, hashes, Base64 también tienen alta entropía
LLM / encoder	Investigación reciente	Contexto semántico, F1 > 0,95	Enviar secretos a una API externa introduce el mismo riesgo

Brecha identificada: ninguna herramienta combina cobertura alta + precisión alta + clasificación *local* sin dependencia de APIs externas.

3. Solución propuesta



¿Qué lo diferencia?

- Clasificador **local**: sin API externa (sin nueva superficie de ataque)
- Contexto del fichero como señal: ¿está en un directorio de *test*? ¿hay un comentario de ejemplo?
- Integrable como *pre-commit hook* o en CI/CD

Stack técnico

- Python 3.12 · scikit-learn
- Random Forest (300 árboles)
- Reproducible: seed=42
- Sin GPU · 15 GB RAM

4. Desarrollo y decisiones clave

Corpus propio (Plan B) — SecretBench inaccesible

900 instancias etiquetadas · 400 secretos reales · 500 falsos positivos difíciles
Diseño anti-trivial: FP con formato válido de proveedor, pero en contexto de *test*

3 iteraciones del corpus Bug crítico detectado

- **v1:** $F1 = 1,00$ → artefacto de construcción
- **v2:** $F1 = 0,62$ → demasiado difícil, Recall destruido
- **v3:** $F1 = 0,95$ → equilibrio honesto ✓
- 2 features añadidas al N1 no registradas en el N2
- Clasificador *ciego* a las señales más importantes
- $F1\ 0,62 \rightarrow 0,95$ al corregirlo

Decisión: Random Forest en lugar de CodeBERT

Hardware CPU-only sin GPU · Corpus < 1.000 instancias · Convergencia en segundos · Interpretable mediante importancias · Generalización equivalente en el dominio

5. Resultados

Sistema	Precision	Recall	F1-score
TruffleHog v3.96	0,48	0,20	0,28
GitLeaks v8.30	0,65	0,76	0,70
N1 — Detector base (regex + entropía)	0,57	0,99	0,72
N2 — N1 + Random Forest ★	0,98	0,93	0,95

+25pp

mejora F1 vs GitLeaks
(0,70 → 0,95)

300→3

falsos positivos
(N1 corpus completo →
N2 test)

CV 5-fold: $F1 = 0,964 \pm 0,010$

Validación en repos reales

trufflesecurity/test_keys

P = 1,00 · R = 1,00 · F1 = 1,00

0 falsas alarmas en código de
seguridad real

6. Conclusiones y trabajo futuro

Lo aprendido

- El **contexto supera al contenido**: ¿está en un fichero de test? es más informativo que la entropía del token
- Un corpus bien diseñado es tan crítico como el modelo
- La paridad entre módulos de extracción y clasificación es un punto de fallo frecuente

Limitaciones honestas

- Corpus sintético: no replica la diversidad de SecretBench (97k instancias reales)
- N2 depende de convenciones de nomenclatura de directorios
- No detecta credenciales sin comillas ni URLs con *basic auth*
- No valida si la credencial sigue activa

Trabajo futuro

- ① Evaluar con SecretBench real
- ② Fine-tuning CodeBERT sobre corpus real
- ③ Ampliar catálogo: credenciales sin comillas + basic auth URLs